An Efficient Agent Location Management for Wireless Sensor Networks

Hiroaki Fukuda
Information Science and Engineering,
Shibaura Institute of Technology
Email: hiroaki@shibaura-it.ac.jp

Paul Leger
Escuela de Ciencias Empresariales
Universidad Católica del Norte, Chile
Email: pleger@ucn.cl

***Abstract*—A Wireless Sensor Network (WSN) is typically deployed on a place in which no electric source is provided, meaning that its battery consumption is crucial. Applications for WSNs require implementations of complex operations like network administration. To simplify the development of these applications, several mobile agent middlewares have been proposed (*e.g.,* Agilla). Applications for these middlewares are executed by communications among agents; therefore, a common operation is to look up agents. Because existing proposals do not have much technical support for efficient approach to look up agents, every lookup consumes a significant amount of battery energy and time. In addition, current approaches can fail their lookup operations if the target agent moves during a lookup operation. This paper proposes CMSN, an efficient and effective lookup for mobile agent middlewares using distributed hash table. We evaluate and compare CMSN in terms of effective lookups and battery consumption.**

***Keywords*-Distributed Hash Table; Mobile Agent; Efficient Lookup;**

I. Introduction

Wireless Sensor Networks (WSNs) consist in a number of sensor nodes[1] used to detect and react to events. WSNs are applied in various domains such as environment observations [2]. Due to the embedded pervasive nature of WSNs, each node is usually deployed on a place in which no electric source is provided or is with difficulty (*e.g.,* forest, mine). Therefore, each node has to use a battery, which must work for a long time (*e.g.,* years). In a node, each operation consumes energy and a WSN must execute several operations on their nodes (*e.g.,* network connections). To simplify the use of these operations, several mobile-agent middlewares have been proposed to allow developers to build applications over a simplified abstraction layer [2]. However, these middlewares do not provide much technical support for an efficient lookup of agents. As a consequence, an agent has to look up the location of its target agent in an ad-hoc manner: basically visiting every node. This lookup approach consumes a significant amount of the battery. In addition, existing lookup approaches for these middlewares do not always work correctly. For example, if the target agent moves during the lookup operation, this agent could not be found. This paper proposes CMSN, an efficient and effective agent lookup for mobile agent middlewares in WSNs. CMSN is based on CSN [1], a distributed hash table algorithm to look up agents. As a result, it is possible to save battery energy of nodes in a WSN when a lookup operation is executed. In addition, CMSN can find a target even if it is moving. To validate this proposal, we provide the prototype implementation of CMSN and show improvements on effective lookups and battery consumptions.

[1]For now, we will use the term "node" to refer to "sensor node".

II. Lookup of Agents in Wireless Sensor Network

In current mobile agent middlewares like Agilla, programmers need to write lookup operations by themselves. Naive solutions to carry out this operation are to randomly visiting nodes or using flooding, leading to two shortcomings:

Migrations during lookups. This solution does not always work because the target agent can migrate to another node during the lookup operation.

Battery consumption. In each visit, a node executes a local lookup operation to find the target agent, which consumes the battery energy.

III. An Efficient Agent Lookup Approach

CMSN uses Distributed Hash Table (DHT) algorithms. Although there have been existing proposals that introduce DHT to route a request to destinations, no proposal considers agent migration. Meanwhile, agent middlewares like Agilla assume that agents are added or removed dynamically, however the number of node in a WSN is fixed and nodes never move. CMSN makes use of this assumption to provide a simple effective lookup of agents on the fly. As CMSN is inspired by CSN [1], CMSN introduces hierarchical ring structures in order to find the target agent with $O(MlogN)$[2]. The DHT algorithm basically consists of two issues: Assigning Identifiers and Routing requests. In addition, CMSN can handle agent migration for mobile agent based middlewares. We explain these issues as follows.

[2]N is the number of nodes that joins the highest level and M is the maximum path length of the energy efficient path between nodes + 1 [1].

978-1-4799-8856-3/15 $31.00 © 2015 IEEE
DOI 10.1109/DCOSS.2015.19

IEEE computer society

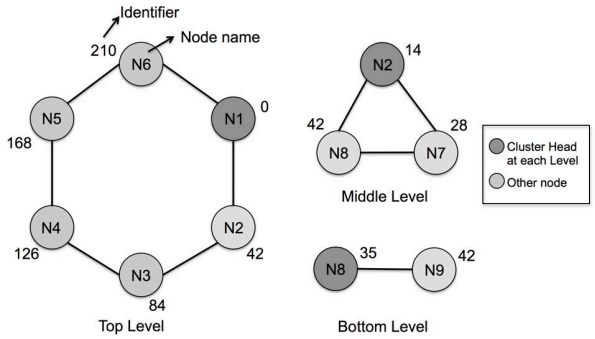

Figure 1: Identifier assignments at three levels.

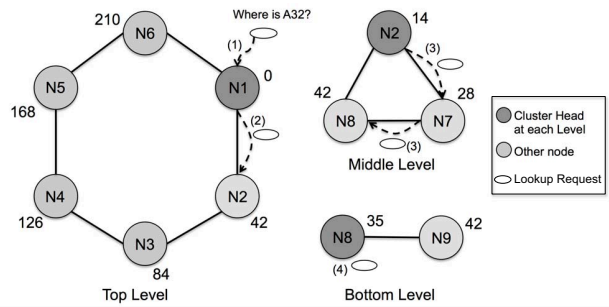

Figure 2: Illustration of lookup behavior steps in CMSN.

A. Assigning Identifiers in CMSN

Although CSN uses the same hash function (*e.g., SHA1*) to assign an identifier to each node and agent, CMSN uses a hash function only for agent identifiers. Node identifiers are assigned by the following three equations:

$$scope(l) = \begin{cases} \frac{Max(\text{hash})}{N(l)} & (l = 0) \\ \frac{scope(l-1)}{N(l)} & (otherwise) \end{cases} \quad (1)$$

$$node_id_l(i) = node_id_l(i-1) + scope(l) \quad (2)$$

$$node_id_l(0) = \begin{cases} 0 & (l = 0) \\ scope(l) + OF_{l-1} & (otherwise) \end{cases} \quad (3)$$

In these equations, l is a ring level – assuming that $l = 0$ corresponds to the top level where the identifier assignment begins. $Max(hash)$ is the maximum number of the hash function that is used. $N(l)$ is the number of nodes per ring that is manually defined as explained above. By using these variables, $scope(l)$ is calculated as equation (1). $scope(l)$ represents a unit of boundary that each node needs to manage in a ring. Because the node assignment process starts from a cluster head per ring where a node identifier is defined by inductive manner as shown in equation (2), and $node_id_l(0)$ in equation (3) always refers to a cluster head at each level. In equation (3), OF_{l-1} represents the identifier of a node that is a predecessor of the node in the upper level.

Figure 1 illustrates an assigning identifier example with a hash function of 8 bits. The figure shows three levels of rings: top, middle and bottom. Each level has a cluster head: node N1 for the top level, and node N2 for the middle and bottom level. According to the equation (1), $N(0)$ is 6 and $Max(0)$ is $2^8 - 1$, $scope(0)$ is 42. From equations (2) and (3), the identifier of N1 and its successor (*i.e.,* N2) at the top level are 0 and 42 respectively. At the middle level, $N(1)$ is 3 manually assigned, and $scope(1)$ is calculated to 14 (42/3). Note that although the identifier of N2 at the top level is 42, its identifier at the middle level is calculated again by equation (3). From *otherwise* case in the equation (3), OF is 0 because the predecessor of N2 at the top level is N1, the identifier of which is 0. As a result, the identifier of N2 at the middle level is calculated to 14 (14 + 0) Finally, the identifiers of N7 and N8 at the middle level are calculated 28 and 42 by equation (2). By repeating this algorithm, unique identifiers are assigned to every node on WSNs.

B. Lookup Behavior

Lookup operations of CMSN are based on Chord, which mainly executes *a*) a lookup operation in the local hash table of a node if the target identifier (*e.g.,* agent id) is less than the node identifier or *b*) forward the request to its successor. Similarly to CSN, a cluster head has at least two successors; thereby, a cluster head needs to choose to which successor it must forward the received request when forwarding. The concrete situation of this decision is described at step 3 in the following example.

Figure 2 illustrates the process of a lookup operation. This example starts when a node N1 receives a lookup request for an agent whose identifier is 32. The numbers in this figure correspond to the following numbered items:

1) ***Receive a lookup request from an agent.*** In this example, node N1 receives a lookup request for an agent whose identifier is 32 (A32).
2) ***Forward the request to the same level ring.*** As described in the assigning identifier process (Section III-A), N1 knows that its rings of the middle and bottom level manage identifiers from 211 to 255, meaning that A32 is not managed by N1 and its lower rings. Thereby, N1 forwards the request to its successor (*i.e.,* N2) at the top level.
3) ***Forward the request to the lower level ring.*** N2 compares its identifier at the top level (*i.e.,* 42) with 32, meaning that the location of the agent must be managed by N2 or its bottom rings. Because the identifier of N2 at the middle level is 14, which is less than 32, N2 forwards the request to the successor at the middle level (*i.e.,* N7). For the same reason, N10 forwards the request to N8.
4) ***Answer the location of an agent or NotFound.*** The identifier of N8 at the middle level is 42, therefore

Approach	Lookups	Failed lookups	Failed lookup rate
CMSN	500	0	0.0%
Random Walk	500	31	6.2%
Flooding	500	5	1.0%

Table I: Comparison of failed lookup rates of CMSN, Random Walk, and Flooding.

Battery capacity: 21,600 Joules.		
Approach	DHT creation	Per lookup
CMSN	1.556 (0.0072%)	0.0049 (0.226×10^{-4}%)
Random Walk	0 (0%)	0.1170 (5.415×10^{-4}%)
Flooding	0 (0%)	0.0432 (1.998×10^{-4}%)

Table II: Comparison of the battery consumption between CMSN, Random Walk, and Flooding.

N8 and its bottom ring must manage the location of A32. The identifier of N8 at the bottom level is 35, therefore N8 is the candidate that must manage the identifier of the request. Finally, N8 replies the location of A32 (*i.e.,* node identifier) if the local hash table contains the location, otherwise N8 returns NotFound message.

C. Agent Migration

As described in section II, if an agent migrates to another node during a lookup operation, this lookup will fail. To prevent this situation from occurring, we apply an approach similar to Mobile Internet Protocol version 6 (MIPv6). As an agent finds a location of another agent (A1) using a lookup operation, A1 can also look up the node that manages its own location in order to block repling an answer against a lookup request. To prevent lookup failures during agent migration, an agent sends an UnderMigration message to a node which manages its location. For example, suppose an agent (A2), which stays in a node (N10), will migrate to a node (N20), which location is managed by a node (N15). In this case, A1 sends an UnderMigration message to N15 before its migration and announces its new location (*i.e.,* N20) to N15 after arriving at N20.

IV. Evaluation

This section shows that CMSN addresses problems previously presented: migrations during lookup operations and battery consumptions by comparing to Random Walk and Flooding. For this evaluation, we use a grid formation that composed of a 12×12 nodes and create three level rings: top, middle and bottom.

Table I, Table II and Figure 3 show the results corresponding to two aspects. Regarding migration during lookup as shown in Table I, the failed lookup rate of CMSN is 0%, while that of random walk and flooding were 6.2% and 1% respectively. This result means that lookup requests of Random Walk and Flooding may return "agent not found" when an agent is a WSN.

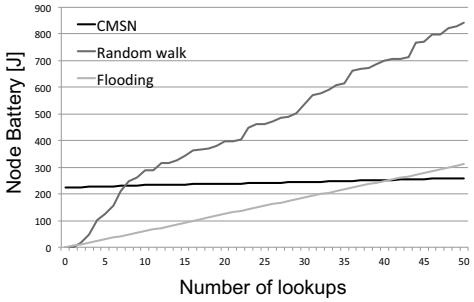

Figure 3: Comparison of CMSN to Random Walk and Flooding.

Regarding battery consumption shown in Table II and Figure 3, although CMSN consumes to create DHT structures, this only consumes 0.0072% of the battery of each node to create them. In addition, CMSN consumes 0.226×10^{-4}% of the battery per lookup operation. In contrast, Random Walk and Flooding consume 5.5415×10^{-4}% and 1.998×10^{-4}% of each node respectively. This result means that Random Walk and Flooding consume less battery when the number of lookup operation is less than 7 times and 41 times respectively. However, when the number of lookup operation increases to 7 or 41, CMSN consumes less battery.

V. Conclusion

Because WSNs promise useful solutions in the real world, it is a motivation to address issues like network and distributed operations. In addition, these applications must execute operations efficiently because nodes of a WSN commonly use batteries to work. To address these issues in an efficient manner and simplify the development of WSN applications, different middlewares have been proposed. One of them is based on agent interactions. For this kind of middleware, an agent must know the exact location of its target agent beforehand to interact. However, existing middlewares, including Agilla, do not provide much technical support to efficiently look up agents. An inefficient implementation of an agent lookup consumes battery, which is crucial for a WSN. In addition, agent lookup operations of mobile agent middlewares could not always return the correct agent location when migration is supported. This paper proposes CMSN that can overcome current shortcommings and its evaluations.

References

[1] M. Ali and Z. A. Uzmi, "CSN: A network protocol for serving dynamic queries in large-scale wireless sensor networks," in *Proceeding of the Second Annual Conference on Communication Networks and Services Research*, 2004, pp. 165–174.

[2] F. Chien-Liang, G.-C. Roman, and C. Lu, "Agilla: A mobile agent midleware for self-adaptive wireless sensor networks," *ACM Transactions on Autonomous and Adaptive System*, vol. 4, no. 3, pp. 1–26, 2009.